

Security Engineering

Lab assignment : Flask Authorization

In the previous Flask tutorials, you developed the Thoughts web application, in which authorization policies were manually implemented. Indeed, the security policies like these can be implemented more systematically, for example, by using an existing Flask authorization library.

In this lab assignment, you will familiarize yourself with several libraries that support enforcing authorization policies and then use these libraries to implement security measures for the Thoughts application. The assignment will also cover how to handle personal data in compliance with GDPR, focusing on the principles of purpose limitation and consent.

Lab assignment: Flask Tutorial 1 and 2

Before starting this assignment, make sure that you have completed the two tutorials on Flask available on the Moodle page. Reminder: You can get help for this assignment in the lab sessions every *Friday, 10:15-12:00, in CHN D 46*.

Contents

Introduction	2
Implementing role-based access control manually	2
Flask-User: Role-based access control	3
Flask-Principal	5
Flask-Oso	7
Purpose-limitation	10
Consent	11

Introduction

In this assignment, we will continue to use the Thoughts application as our running example. Thoughts is a simple web application which allows Persons to create Thoughts, share them with others, and vote for their favorite thoughts. Consider the following authorization policies:

- (SP1) a logged-in user has a USER role;
- (SP2) a user can change only their own personal data and thoughts;
- (SP3) a user can vote at most three times for the same thought; and
- (SP4) a user is not allowed to vote for his own thoughts.

As a starting point, we provide the Thoughts application with all functionalities and authorization policies manually implemented. Please download the FlaskAuth.zip file from the course's webpage.

Implementing role-based access control manually

The base implementation can be found in a sub-directory named /phil-base within the downloaded archive. We will refer to this implementation as the *base implementation*. This *base implementation* is the same as the solution of the previous Flask assignment with some minor changes (see the code snippet below).

```
...
+ class Person(db.Model, UserMixin):
+     def isUser(self):
+         return self.myRole == Role.USER;
...
+ def check_user(caller):
+     if not caller.isUser:
+         raise SecurityException("You are not allowed to access this page: Not a User")
...
+ @app.route('/error')
+ def error():
+     msg = "You are not allowed to access this page: Not a User"
+     return render_template('error.html', message = msg)
...
@app.route('/')
@login_required
def index():
    try:
+         check_user(current_user)
        thoughts=Thought.query.all()
        return render_template('index.html', table=thoughts)
+     except SecurityException as se:
+         return redirect(url_for('error'))
```

Namely, we include an explicit authorization check (the `check_user` function) in each endpoint accessible only to logged in users (exemplified on `index` endpoint below). This check verifies that the logged in user has the role `USER`, as opposed to the `OTHER` role (The previous solution tacitly assumed that all users have the `USER` role, i.e., the `OTHER` role is never used). In the following example, only the person with `USER` role can access the `index`

endpoint. If a user without this role attempts to access this endpoint, then he/she will be redirected to the error endpoint.

Next, we review the main features of several existing libraries that support enforcing authorization policies in Flask applications, and if feasible, demonstrate their usage on the (SP2) policy. We leave the implementation of the other policies for you as an exercise.

Flask-User: Role-based access control

In the last Flask tutorial, we have introduced this library for user authentication and credential management features. In addition to that, Flask-User also provides other security features, including role-based access control (RBAC). Similarly, the following libraries also provide support for the same class of authorization: Flask-RBAC ¹, Flask-Authorize ².

Installation The Docker image we have provided in the base implementation comes with Flask-User pre-installed. To install it locally, activate the virtual environment and then run

```
pip install flask-user==1.0.2.2
pip install email_validator==1.3.0
```

Example We start from the base implementation. Since all of the aforementioned policies (i.e. (SP2)-(SP4)) apply to the users with USER role, we will enforce these requirements using Flask-User.

First, let us demonstrate how to create a Role class and associate it with the current user class. We start this step by modifying the current code snippet in the application:

```
...
app = Flask(__name__)
...
- class Role(Enum):
-     USER = auto()
-     OTHER = auto()
+ USER = "User"
+ OTHER = "Other"

db = SQLAlchemy(app)

+ class Person(db.Model, UserMixin):
+     id = db.Column(db.Integer, primary_key=True)
+     ...
-     myRole = db.Column(db.Enum(Role), default=Role.USER)
+     roles = db.relationship('Role', secondary='user_roles')

+ class Role(db.Model):
+     id = db.Column(db.Integer, primary_key=True)
+     name = db.Column(db.String(50), unique=True)
```

¹Flask-RBAC documentation: <https://flask-rbac.readthedocs.io/en/latest/>.

²Flask-Authorize documentation: <https://flask-authorize.readthedocs.io/en/latest/>.

```

+ class UserRoles(db.Model):
+     id = db.Column(db.Integer, primary_key=True)
+     user_id = db.Column(db.Integer, db.ForeignKey('person.id'), unique=True)
+     role_id = db.Column(db.Integer, db.ForeignKey('roles.id'))
+ ...

with app.app_context():
    db.create_all()
+     roles = Role.query.all()
+     if len(roles) == 0:
+         db.session.add(Role(name=USER))
+         db.session.add(Role(name=OTHER))
+         db.session.commit()

```

In this case, instead of defining an enum class, we define a proper Role class: A role contains an id and a name. By convention of Flask-User library, we create also a UserRoles association between User and Role classes, and we add the constraint that a user can only have one role (i.e. making the association-end unique). Note that with this approach, one also has to initialize the Role objects when the database is first created.

Next, in order to enforce the role-based authorization policy, we include the `@roles_required` annotation over target routes, assign our default error endpoint for handling unauthorized access, and assign appropriate User role when a new user is registered. In our example, we remove the manual `check_user` check with the `@roles_required(USER)` annotation.

```

- from flask_user import login_required, UserManager, UserMixin, current_user
+ from flask_user import login_required, UserManager, UserMixin, current_user, \
+     roles_required, user_registered
+ ...
+ app.config['USER.UNAUTHORIZED_ENDPOINT'] = 'error'
+ ...
+ @user_registered.connect_via(app)
+ def _after_user_registered_hook(sender, user, **extra):
+     role = Role.query.filter_by(name=USER).one()
+     user.roles.append(role)
+     db.session.commit()
+ ...
+ @app.route('/')
+ @login_required
+ @roles_required(USER)
+ def index():
-     try:
-         check_user(current_user)
-         thoughts=Thought.query.all()
-         return render_template('index.html', table=thoughts)
-     except SecurityException as se:
-         return render_template('error.html', message = msg)

```

This approach is handy for application in which role-based access control is required. Code is simpler and easier to maintain as one does not need to explicitly call the `check_user` function and repeat the exception handling.

In conclusion, Flask-User is clearly not expressive enough to enforce the required authorization policies. Therefore, let us consider the next library that provides better support.

Flask-Principal

Flask-Principal³ is a more expressive authorization framework for Flask web applications. Besides RBAC it allows definition of programmatic authorization checks. These checks are organized as follows: after a user logs in the application computes a set of all the user's access rights, called *Needs*, and associates them with an instance of an *Identity* that represents the user. In Flask-Principal, a Need is the smallest unit of access control, e.g., "has the USER role", or "can change their own personal data". A Need is represented as a tuple, e.g., the (role, USER) pair (corresponding to the "has the USER role"), or the (name, change, 42) triple (corresponding to the "can change name of the user object with id 42"). Essentially, Needs are designed to be flexible and typically are domain-specific, though Flask-Principal also provides pre-defined Need types for the common access control requirements. A Permission is a set of Needs, all of which user's Identity needs to have in order to obtain access to a resource.

Installation To install Flask-Principal via Docker, include the following line inside the requirements.txt under the Flask packages section:

```
flask-principal==0.4.0
```

To install it locally, activate the virtual environment and then run

```
pip install flask-principal==0.4.0
```

The following code snippet exemplifies Flask-Principal:

```
from flask_principal import Principal, Identity, Permission, UserNeed, RoleNeed,
    identity_loaded, identity_changed
...
from collections import namedtuple
from functools import partial
from flask_login import user_logged_in

principals = Principal(app)

ThoughtNeed = namedtuple('Thought', ['method', 'value'])
DeleteThoughtNeed = partial(ThoughtNeed, 'delete')
class DeleteThoughtPermission(Permission):
    def __init__(self, tid):
        need = DeleteThoughtNeed(tid)
        super(DeleteThoughtPermission, self).__init__(need)

@user_logged_in.connect_via(app)
def _after_login_hook(sender, user, **extra):
    identity_changed.send(app, identity=Identity(user.id))

@identity_loaded.connect_via(app)
def on_identity_loaded(sender, identity):
    identity.user = current_user
```

³<https://pythonhosted.org/Flask-Principal/>

```

if hasattr(current_user, 'id'):
    identity.provides.add(UserNeed(current_user.id))

if hasattr(current_user, 'myRole'):
    identity.provides.add(RoleNeed(current_user.myRole))

if hasattr(current_user, 'created'):
    for t in current_user.created:
        identity.provides.add(DeleteThoughtNeed(t.id))
...
@app.route('/delete_thought')
@login_required
def delete_thought():
    id = request.args["id"]
    try:
        i = int(id)
        t = Thought.query.get(i)
        if t != None:
            try:
                permission = DeleteThoughtPermission(i)
                if permission.can():
                    db.session.delete(t)
                    db.session.commit()
            else:
                raise SecurityException("You are not allowed ...")
    except SecurityException as se:
        thoughts = Thought.query.all()
        return render_template('index.html',
                               table=thoughts, security_violation=True, message=se.msg)
    return redirect(url_for('index'))
except:
    return redirect(url_for('index'))

```

After initializing an instance of Principal class, we define the permissions in the form of the DeleteThoughtNeed need. It is a partially applied ThoughtNeed with method delete. We then create a permission class that aggregates needs. In this case only the DeleteThoughtNeed need.

Whenever a user logs in (signaled by the `_after_login_hook` function), we initialize the user's identity object. For all of the user's created thoughts, a DeleteThoughtNeed object is created and added to the identity. Finally, in the `delete_thought` endpoint we call the `can` function of the DeleteThoughtPermission to check if the user has the appropriate need loaded.

Clearly, the framework is expressive to capture the authorization policies we need to implement. However, the level of abstraction is quite low. Now, let's look at a framework that improves this aspect.

Flask-Oso

Flask-Oso is a security framework for building authorization checks in web applications. This framework supports different class of authorization policy, including attribute-based access control, request-based access control. Furthermore, the policies are defined in a file separated from the application code and are written in a declarative language called Polar. Information about this framework and the language can be found at <https://docs.osohq.com/index.html>.

Installation To install it via Docker, include the following line inside the `requirements.txt` under the Flask packages section: ⁴

```
oso==0.26.3
```

To install it locally, activate the virtual environment and then run

```
pip install flask-oso==0.26.3
```

Example The following code listings display how to enforce (SP2) into the base implementation. The implementation of (SP2) using Flask-Oso can be found in a sub-directory named `/phil-oso` in the downloaded archive.

To start with, an `Oso` object is initialized and the user class (i.e., `Person` class) and the class we would like to apply authorization policy (i.e., `Thought` class - since (SP2) is the authorization policy about deleting thoughts) are registered via method `register` of this object. The security artifact `main.polar` is also injected into this object. The detail of this file will be depicted later.

```
+ from oso import Oso, ForbiddenError
...
+ oso = Oso()
+ oso.register_class(Thought)
+ oso.register_class(Person)
+ oso.load_files(["main.polar"])
```

Secondly, to enforce the authorization policy on the `delete_thought()` action, the manual check `check_delete` of the base implementation is replaced with function `oso.authorize`. As depicted in the snippet, this provided function takes four arguments, namely (i) the user who is attempting to perform the action, (ii) the action that he/she is performing, (iii) the object on which the action is attempted to be performed, and (iv) (optional) a boolean flag to decide whether the user must have the read permission as a prerequisite. In this case, for simplicity, we turn off this flag. ⁵ Informally, if the user is not authorized to perform the

⁴Note: For version 0.26.3, the package was not available on ARM architectures. The issue is known (<https://github.com/osohq/oso/issues/1529>) and the workaround is to build the container using the linux/amd64 platform by changing the docker-compose file.

⁵See more about this flag at <https://docs.osohq.com/python/reference/api/index.html#oso.Oso.authorize1>.

action on the object, then a `ForbiddenError` exception will be thrown. For this reason, the `SecurityException` catch statement is replaced with the new `ForbiddenError` one.

```
@app.route('/delete_thought')
@login_required
def delete_thought():
    id = request.args["id"]
    try:
        i = int(id)
        t = Thought.query.get(i)
        if t != None:
            try:
                - check_delete(t, current_user)
                + oso.authorize(current_user, "delete", t, check_read=False)
                db.session.delete(t)
                db.session.commit()
            - except SecurityException as se:
            + except ForbiddenError:
                thoughts = Thought.query.all()
                + msg = "You are not allowed to delete this thought: Not own thought"
                return render_template('index.html', table=thoughts,
                    - security_violation=True, message=se.msg)
                    + security_violation=True, message=msg)
            return redirect(url_for('index'))
    except:
        return redirect(url_for('index'))
```

Finally, the following example showcase how we define (SP2) in the security artifact. In Flask-Oso, policy are defined in a file separated from the application code and are written in a declarative language called Polar.⁶

In a nutshell, this file contains a set of resources - to which represent which classes the security policies will enforce, an actor - to which represent which class will be the user class, and security rules - the set of allow rules that define the conditions when the user with which role is authorized to perform an action over a resource. More concretely, in this case, the `Person` class is the actor and the `Thought` class is a resource.

Furthermore, for every resource, a set of permissions (as strings) and a set of roles (also as strings) are defined. These correspond to the set of actions and roles that are considered within this resource. More concretely, in this case, the resource `Thought` comprises of a permission set $P = \{\text{"delete"}\}$ and a role set $R = \{\text{"user"}\}$.

When `oso.authorize(u, a, res)` from the application is invoked from the application, it will call `allow(u, a, res)` which in turn call `has_permission(u, a, res)`. Here, it checks both (i) whether u has permission to perform a on res — in this case, u is the owner of res — and (ii) whether u has any role $r \in R$ — in this case, u has the `USER` role.

⁶Learn more about Polar syntax at <https://docs.osohq.com/any/reference/polar/polar-syntax.html>.

```
actor Person {}

resource Thought {
  permissions = ["delete"];
  roles = ["user"];
}

allow(actor, action, resource) if
  has_permission(actor, action, resource);

has_permission(p: Person, "delete", t: Thought)
  if is_owner(p, t) and has_role(p, "user", t);

has_role(p: Person, "user", _: Thought) if p.isUser();

is_owner(p: Person, t: Thought) if p.id == t.createdBy.id;
```

Recently, Amazon introduced Cedar⁷ – an open-source policy language for access control. In Cedar, you can define access control policies as set of different permit or deny rules. Similar to Flask-Oso, these rules are typically defined in a separate file from the application code.

Since there is no direct support for embedding the Cedar authorization engine into the Flask application, we do not intend to showcase Cedar as one of the approach in this context. However, students who are interested are welcome to explore the approach and implement authorization using Cedar.

Lab exercise: Implement the (SP3) and (SP4) policies

Now you should use these approach to implement the remaining two authorization policies, i.e., (SP3) and (SP4).

⁷Cedar: <https://www.cedarpolicy.com/en/tutorial>

Purpose-limitation

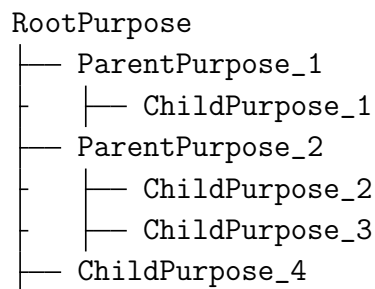
From the General Data Protection Regulation (GDPR), Purpose-limitation (Art. 5 §1 (b)) states that “Personal data shall be collected for specified, explicit and legitimate purposes and not further processed in a manner that is incompatible with those purposes”. In this exercise, we consider the base implementation. Recall, the base implementation is located in a sub-directory named `/phil-base` within the downloaded archive.

Personal data Consider the Person’s attributes, such as username and role; association-ends, such as created and voted, are personal data.

Purposes In what follows we define the set of purposes P .

$P = \{\text{RootPurpose}, \text{ParentPurpose}_1, \text{ParentPurpose}_2, \text{ChildPurpose}_1, \text{ChildPurpose}_2, \text{ChildPurpose}_3, \text{ChildPurpose}_4\}$.

Consider the following purposes and their purpose hierarchy:



Actual purposes Assuming the `delete_thought()` function is associated with the purpose `ChildPurpose_3`, `vote_thought()` is associated with the purpose `ParentPurpose_2` and `check_user()` is associated with the purpose `ChildPurpose_4`.

Do you think the base implementation with the following privacy notice complies with the privacy requirements of “purpose limitation” from the GDPR?

Declared purposes

The personal data (username, role, created, voted) will be processed for the following purposes:

1. The username and role of a person are read for the purpose `RootPurpose`.
2. The votes of a person (i.e., voted association-end) is read for the purpose of `ParentPurpose_2`.
3. The thoughts of a person (i.e., created association-end) is read for the purpose of `ChildPurpose_2`.

Consent

From the General Data Protection Regulation (GDPR), Data-subject consent (Art. 7 §1) states that “Where processing is based on consent, the controller shall be able to demonstrate that the data subject has consented to processing of his or her personal data.”.

To do so, we need to extend our database schema to support storing users' consents of processing their personal data. Assuming that personal data belongs solely to the owner of the data (i.e., the data subject is the person per se).

How would you extend the current base implementation to support the above privacy requirements?